МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
Кафедра інформаційних технологій

ЛАБОРАТОРНА РОБОТА №2

З дисципліни: «Об’єктно-орієнтоване та подійно-орієнтоване програмування»

На тему: «Основи роботи з класами та об’єктами. Інкапсуляція. Конструктор. Модифікатори доступу»

Роботу виконав:
студент II курсу групи АД-251
спеціальності F6
Горбунова А.А.

Роботу перевірив:
к т.н., доцент Рудніченко М.Д.

Оцінка: _____

ДУОП, 2026

1 МЕТА РОБОТИ ТА ЗАВДАННЯ

Мета роботи:

1) Ознайомитись з основами написання класів;
2) Освоїти інкапсуляцію класів;
3) Розібратися із використанням конструктора;
4) Навчитися використовувати модифікатори доступу.

Завдання на лабораторну роботу:

1) **Створіть клас «Товар» (`Item`).**

А. Клас «Товар» має містити такі поля: найменування (рядок), ціна (`float`).

Б. Клас «Товар» повинен містити конструктор, який приймає два параметри: найменування та початкову ціну товару.

В. Клас «Товар» повинен мати такі публічні методи: підвищення ціни на певний відсоток (значення відсотка типу `float` передається як аргумент методу); зниження ціни на певний відсоток (значення відсотка типу `float` передається як аргумент методу).

Г. У класі має бути реалізована перевірка ціни на негативне значення. Якщо в конструкторі передається негативне значення ціни, чи в результаті зниження ціни на $> 100\%$ ціна стає негативною, вона має бути примусово встановлена в 0.

2) **Створіть клас «Кошик» (`Cart`).**

А. «Кошик» має реалізовувати структуру даних стек, у якому містяться об'єкти класу «Товар». Вважатимемо, що ми додаємо завжди по 1 одиниці товару. Стек «всередині» реалізується традиційним масивом об'єктів `Item`. Клас повинен містити перевірки, пов'язані з роботою стека — переповнення стека, спроба витягти елемент із порожнього стека і т.д.

Б. «Кошик» повинен містити конструктор з 1 параметром — максимальною кількістю елементів у стеку. У цьому конструкторі повинна відбуватися ініціалізація

масиву, який реалізує стек.

В. «Кошик» повинен містити такі публічні методи: додавання товару, видалення товару, підрахунок суми цін товарів у кошику (пройтися за елементами масиву та скласти всі значення цін), підвищення та зниження цін усіх товарів у стеку (два окремі методи, значення відсотка передається як параметр методу, необхідно пройтись по всіх елементах масиву та передати відповідне повідомлення об'єктам).

3) **У методі `Main` необхідно створити об'єкт класу «Кошик» з деякою максимальною кількістю елементів у стеку.**

А. Заповнити кошик об'єктами класу `Item` (5–6 об'єктів буде достатньо);

Б. Вивести суму цін товарів усередині кошика;

В. Підняти ціни в кошику на 15%, вивести змінену суму цін на консоль;

Г. Знизити ціни в кошику на 30%, вивести змінену суму цін на консоль.

4) **Перепишіть клас `Cart`, використовуючи замість стека чергу (або колекцію, якщо ви знаєте, що це таке) без зміни інтерфейсу класу. Перевірте роботу коду.**

5) **Додаткове індивідуальне завдання з файлу лаб2-2.docx. Варіант №7**

Облік товарів на складі — Створіть просту консольну програму для обліку товарів. Програма повинна працювати з масивом назв товарів та відповідним масивом їхньої кількості. Необхідно вивести всі товари, визначити товар із найбільшою кількістю одиниць, підрахувати загальну кількість товарів на складі та знайти товари, кількість яких менша за задане користувачем мінімальне значення. Для перебору масивів використайте цикли, а для перевірки кількості — умовні конструкції.

2 ТЕОРЕТИЧНІ ОСНОВИ ООП: ІНКАПСУЛЯЦІЯ ТА МОДИФІКАТОРИ ДОСТУПУ

2.1 Концепція інкапсуляції та приховання реалізації

Об’єктно-орієнтоване програмування базується на представленні програми як сукупності взаємодіючих об’єктів. Одним із фундаментальних принципів цієї парадигми є **інкапсуляція** — процес відокремлення один від одного елементів об’єкта, що визначають його внутрішній устрій (реалізацію) від Його зовнішньої поведінки (інтерфейсу).

У межах виконання лабораторної роботи інкапсуляція розглядається як захисна оболонка, яка оберігає дані об’єкта від неконтрольованого прямого втручання з боку зовнішнього коду. Взаємодія з об’єктами здійснюється виключно через чітко визначений публічний контракт методів (інтерфейс), що дозволяє локалізувати частини реалізації системи, які можуть зазнати змін у майбутньому (наприклад, повна заміна внутрішньої структури кошика зі стека на чергу без зміни зовнішнього коду додатку).

2.2 Модифікатори доступу в мові Java

Для практичної реалізації механізму інкапсуляції в мові Java використовується система рівнів захисту полів та методів класів за допомогою модифікаторів доступу:

– **`private`:** Найсуворіший рівень доступу. Члени класу, оголошені з цим модифікатором, доступні лише методам та конструкторам всередині цього ж класу. Закриття полів (`name`, `price`, `stack`) забезпечує повний контроль над станом об’єкта.

– **За замовчуванням (`package-private`):** Рівень доступу, який застосовується автоматично за відсутності модифікатора. Елементи доступні будь-яким іншим класам, що знаходяться у тому самому пакеті.

– **`protected`:** Забезпечує видимість членів класу для всіх компонентів всередині поточного пакету, а також для підкласів (нащадків) в інших зовнішніх пакетах.

– **`public`:** Повністю відкритий доступ. Члени класу доступні для будь-яких зовнішніх методів та класів як у поточному, так і в інших пакетах додатка. Складають зовнішній інтерфейс взаємодії об'єкта.

2.3 Модифікатори доступу та анотація @Override в архітектурі інтерфейсів

Особливе місце в реалізації об'єктно-орієнтованого поліморфізму посідає службовий маркер компілятора — анотація **`@Override`**. На відміну від мови C++, де для створення віртуальних функцій у базовому класі обов'язково використовується ключове слово `virtual`, у Java всі нестатичні методи є віртуальними за замовчуванням.

У розроблених компонентах `StackCart` та `QueueCart` анотація `@Override` застосована перед кожним методом, що приходить із контракту інтерфейсу `Cart`. Функціональне призначення цієї директиви полягає в наступному:

1) **Контроль сигнатури:** Вона примусово наказує компілятору перевірити, чи дійсно метод з такою самою назвою та параметрами існує в інтерфейсі.

2) **Захист від помилок розробника:** Якщо під час написання коду в дочірньому класі буде допущено друкарську помилку (опечатку) у назві методу, компілятор зупинить збирання проєкту з помилкою невідповідності, що запобігає виникненню прихованих багів непрацюючого поліморфізму.

2.4 Роль конструкторів та ключового слова `this`

Конструктор є спеціальним методом класу, який автоматично викликається оператором `new` під час створення нового екземпляра для ініціалізації його поча-

ткового стану. Конструктор не має типу повертаємого значення, а його ім’я строго збігається з назвою класу.

У розроблених компонентах (`Item`, `StackCart`, `QueueCart`) імена вхідних аргументів конструкторів збігаються з іменами внутрішніх змінних екземпляра. Для розв’язання конфлікту імен застосовано приховане посилання на поточний об’єкт за допомогою ключового слова **`this`** (наприклад, `this.price = price`), що дозволяє компілятору чітко розрізняти локальні параметри методу від полів об’єкта у кучі (Heap).

2.5 Об’єктно-орієнтована архітектура та UML-діаграма класів

Для детального аналізу взаємодії створених компонентів програми було розроблено архітектурну UML-діаграму класів у середовищі Draw.io. Спроектована структура наочно демонструє застосування ключових принципів об’єктно-орієнтованого програмування, зокрема інкапсуляції та поліморфізму через інтерфейсні контракти.

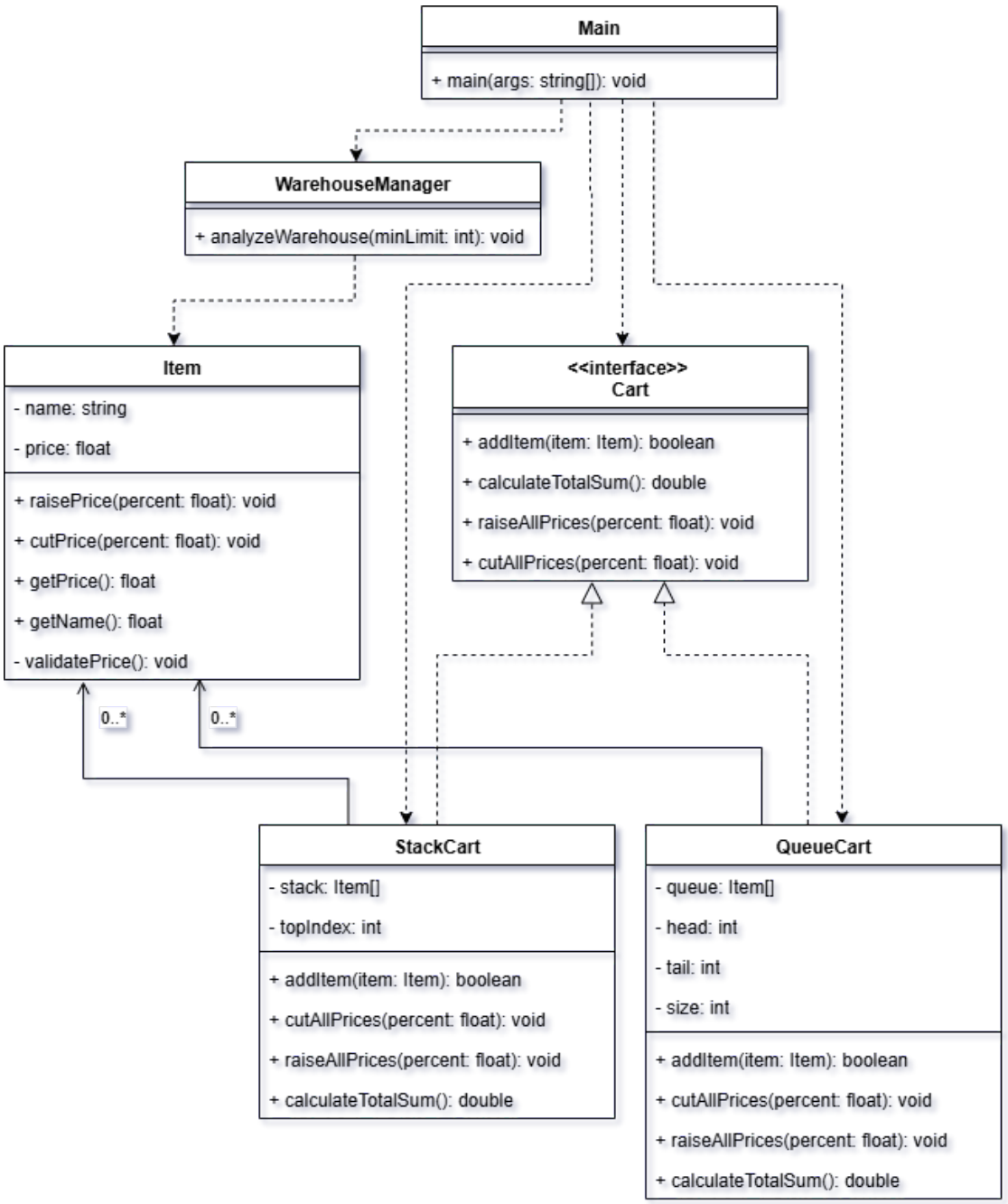


Рисунок 2.1 – UML-діаграма взаємодії класів та інтерфейсів розробленого додатка

Основними архітектурними зв’язками на схемі (рис. 2.1) є:

– **Відношення реалізації (Realization):** Класи `StackCart` та `QueueCart` пов’язані з інтерфейсом `Cart` пунктирними стрілками з білим трикутним наконечником. Це показує, що обидва компоненти зобов’язані повністю виконати публічний контракт методів, приховуючи власну внутрішню логіку (стек або чергу).

– **Відношення асоціації (Association):** Класи кошиків посилаються на сутність

`Item`, оскільки містять всередині себе масиви об’єктів товарів, реалізуючи відношення агрегації типу «один до багатьох» (`0..*`).

– **Відношення залежності (Dependency):** Клас `WarehouseManager` (Індивідуальний варіант №7 з фійлу лаб2-2.docx) та головний файл `Main` використовують класи сутностей для ініціалізації та виконання аналітичних розрахунків.

3 ЛІСТИНГ ПРОГРАМНОГО КОДУ

У цьому розділі наведено повний вихідний код. Код оформлено згідно зі стандартами *Oracle Code Conventions* та містить вичерпну документацію у форматі *JavaDoc*.

3.1 Клас Main.java

```
1  /*
2   * Copyright (c) 2026 Anastasiia Gorbunova
3   * National University "Odesa Polytechnic"
4   *
5   * Licensed under the Apache License, Version 2.0 (the "License");
6   * you may not use this file except in compliance with the License.
7   * You may obtain a copy of the License at
8   *
9   *     http://apache.org
10  *
11  * Unless required by applicable law or agreed to in writing, software
12  * distributed under the License is distributed on an "AS IS" BASIS,
13  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
14  * See the License for the specific language governing permissions and
15  * limitations under the License.
16  */
17 package ad251.gorbunova;
18
19 /**
20  * Головний керуючий клас програми (Точка входу).
21  * Координує створення об'єктів-компонентів та демонстрацію виконання завдань
22  * лабораторної роботи №2.*.
23  *
24  * @author Анастасія Горбунова
25  * @version 1.0 09 Oct 2026
26  */
27 public class Main {
28
29     public static void main(String[] args) {
30         System.out.print("=== Лабораторна робота №2 ===\n\n");
31         System.out.print("Тема: Основи роботи з класами та об'єктами.\n");
32         System.out.print("Інкапсуляція. \n");
33         System.out.print("Модифікатори доступу.\n\n");
34
35         // --- 1. Традиційне виконання на базі Стека (LIFO) ---
36         System.out.print("--- Тестування Кошика на базі СТЕКА (LIFO) ---\n");
37         Cart stackCart = new StackCart(8); /* Створення через інтерферс контракт */
38
39         // Заповнення 5-6 об'єктами
40         stackCart.addItem(new Item("Ноутбук", 25000.0f));
41         stackCart.addItem(new Item("Мишка", 600.0f));
42         stackCart.addItem(new Item("Клавіатура", 1500.0f));
43         stackCart.addItem(new Item("Монітор", 7000.0f));
44         stackCart.addItem(new Item("Навушники", 2200.0f));
45
46         System.out.printf("Початкова сума товарів у стеку: %.2f грн\n",
           ↪  stackCart.calculateTotalSum());
47
48         // Підняття цін на 15%
49         stackCart.raiseAllPrices(15.0f);
50         System.out.printf("Сума після підвищення цін на 15%%: %.2f грн\n",
           ↪  stackCart.calculateTotalSum());
51
52         // Зниження цін на 30%
53         stackCart.cutAllPrices(30.0f);
54         System.out.printf("Сума після зниження цін на 30%%: %.2f грн\n\n",
           ↪  stackCart.calculateTotalSum());
55
56
57         // --- 2. Пункт 4: Зміна внутрішньої структури на Чергу (FIFO) ---
58         System.out.print("--- Тестування Кошика на базі ЧЕРГИ (FIFO) ---\n");
59         Cart queueCart = new QueueCart(8); /* Інтерфейс ТОЙ САМИЙ, логіка інша */
60
```

```
61          queueCart.addItem(new Item("Ноутбук", 25000.0f));
62          queueCart.addItem(new Item("Мишка", 600.0f));
63          queueCart.addItem(new Item("Клавіатура", 1500.0f));
64
65          System.out.printf("Початкова сума у черзі: %.2f грн\n", queueCart.calculateTotalSum());
66          queueCart.raiseAllPrices(15.0f);
67          System.out.printf("Сума у черзі після підвищення на 15%%: %.2f грн\n\n",
            ↪  queueCart.calculateTotalSum());
68
69
70          // --- 3. ЗАДАННЯ з файлу lab2-2.docx: Варіант №7 (Облік товарів на складі) ---
71          System.out.print("--- Індивідуальне завдання (Варіант №7) ---\n");
72          WarehouseManager.analyzeWarehouse(10); /* Передаємо мінімальний ліміт 10 одиниць */
73
74      }
75
76  }
```

3.2 Клас Item.java

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Клас, що описує сутність "Товар".
 * Забезпечує інкапсуляцію даних та безпечну валідацію вартості.
 *
 * @author Анастасія Горбунова
 * @version 2.0 09 Oct 2026
 */
public class Item {
    /** Найменування товару */
    private final String name;
    /** Поточна ціна товару */
    private float price;

    /**
     * Конструктор для ініціалізації товару.
     *
     * @param name найменування товару
     * @param initialPrice початкова вартість
     */
    public Item(String name, float initialPrice) {
        this.name = name;
        setPrice(initialPrice); /* Виклик валідатора для запобігання негативній ціні */
    }

    /**
     * Публічний метод для підвищення ціни на певний відсоток.
     *
     * @param percent відсоток підвищення (значення типу float)
     */
    public void raisePrice(float percent) {
        if (percent > 0) {
            this.price += this.price * (percent / 100.0f);
        }
    }

    /**
     * Публічний метод для зниження ціни на певний відсоток.
     * Автоматично скидає ціну до 0, якщо зниження перевищує 100%.
     *
     * @param percent відсоток зниження (значення типу float)
     */
    public void cutPrice(float percent) {
        if (percent > 0) {
            this.price -= this.price * (percent / 100.0f);
            validatePrice();
        }
    }

    /**
     * Приватний метод-валідатор для контролю негативних значень ціни.
     */
    private void validatePrice() {
        if (this.price < 0.0f) {
            this.price = 0.0f; /* Примусове встановлення в нуль згідно умов завдання */
        }
    }

    // --- Сеттери та Геттери згідно з конвенцією безпеки ---

    public String getName() {
        return name;
    }

    public float getPrice() {
        return price;
```

```
78      }
79
80      public void setPrice(float price) {
81          this.price = price;
82          validatePrice();
83      }
84  }
```

3.3 Інтерфейс Cart.java

```
1  /*
2   * Copyright (c) 2026. Anastasiia Gorbunova
3   * National University "Odesa Polytechnic"
4   *
5   * Licensed under the Apache License, Version 2.0 (the "License");
6   * you may not use this file except in compliance with the License.
7   * You may obtain a copy of the License at
8   *
9   *     http://apache.org
10  */
11 package ad251.gorbunova;
12
13 /**
14  * Інтерфейс кошика товарів. Визначає загальнодоступний контракт (API)
15  * для будь-якої внутрішньої структури даних зберігання.
16  */
17 public interface Cart {
18     /** Додавання товару до кошика */
19     boolean addItem(Item item);
20
21     /** Видалення останнього доданого товару з кошика */
22     Item deleteItem();
23
24     /** Підрахунок суми цін усіх товарів у кошику */
25     double calculateTotalSum();
26
27     /** Підвищення ціни всіх товарів у кошику на відсоток */
28     void raiseAllPrices(float percent);
29
30     /** Зниження ціни всіх товарів у кошику на відсоток */
31     void cutAllPrices(float percent);
32 }
33
```

3.4 Клас StackCart.java

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Реалізація кошика на базі структури даних LIFO (Стек) за допомогою масиву.
 */
public class StackCart implements Cart {
    /** Масив об'єктів для зберігання елементів стека */
    private final Item[] stack;
    /** Індекс вершини стека */
    private int topIndex;

    /**
     * Конструктор кошика-стека.
     *
     * @param capacity максимальний розмір стека
     */
    public StackCart(int capacity) {
        this.stack = new Item[capacity];
        this.topIndex = -1; /* Стек порожній */
    }

    @Override
    public boolean addItem(Item item) {
        // Перевірка на переповнення стека
        if (topIndex >= stack.length - 1) {
            System.out.print("[ПОМИЛКА] Кошик-стек переповнений! Не вдалося додати: " +
            ↪  item.getName() + "\n");
            return false;
        }
        stack[++topIndex] = item;
        return true;
    }

    @Override
    public Item deleteItem() {
        // Перевірка на спробу видалити елемент із порожнього стека
        if (topIndex < 0) {
            System.out.print("[ПОМИЛКА] Спроба видалити елемент із порожнього кошика!\n");
            return null;
        }
        Item removedItem = stack[topIndex];
        stack[topIndex--] = null; /* Очищення посилання для роботи Garbage Collector */
        return removedItem;
    }

    @Override
    public double calculateTotalSum() {
        double sum = 0.0;
        for (int i = 0; i <= topIndex; i++) {
            sum += stack[i].getPrice();
        }
        return sum;
    }

    @Override
    public void raiseAllPrices(float percent) {
        for (int i = 0; i <= topIndex; i++) {
            stack[i].raisePrice(percent); /* Відправка повідомлення об'єкту Item */
        }
    }

    @Override
    public void cutAllPrices(float percent) {
        for (int i = 0; i <= topIndex; i++) {
            stack[i].cutPrice(percent);
        }
    }
```

```
77  }
```

3.5 Клас QueueCart.java

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Реалізація кошика на базі структури даних FIFO (Черга) за допомогою масиву.
 * Демонструє незмінність інтерфейсу при повній зміні внутрішньої логіки.
 */
public class QueueCart implements Cart {
    private final Item[] queue;
    private int head;
    private int tail;
    private int size;

    public QueueCart(int capacity) {
        this.queue = new Item[capacity];
        this.head = 0;
        this.tail = 0;
        this.size = 0;
    }

    @Override
    public boolean addItem(Item item) {
        if (size == queue.length) {
            System.out.print("[ПОМИЛКА] Кошик-черга переповнений!\n");
            return false;
        }
        queue[tail] = item;
        tail = (tail + 1) % queue.length; /* Кільцевий буфер */
        size++;
        return true;
    }

    @Override
    public Item deleteItem() {
        if (size == 0) {
            System.out.print("[ПОМИЛКА] Кошик-черга порожній!\n");
            return null;
        }
        Item removedItem = queue[head];
        queue[head] = null;
        head = (head + 1) % queue.length;
        size--;
        return removedItem;
    }

    @Override
    public double calculateTotalSum() {
        double sum = 0.0;
        int current = head;
        for (int i = 0; i < size; i++) {
            sum += queue[current].getPrice();
            current = (current + 1) % queue.length;
        }
        return sum;
    }

    @Override
    public void raiseAllPrices(float percent) {
        int current = head;
        for (int i = 0; i < size; i++) {
            queue[current].raisePrice(percent);
            current = (current + 1) % queue.length;
        }
    }

    @Override
    public void cutAllPrices(float percent) {
        int current = head;
```

```
78          for (int i = 0; i < size; i++) {
79              queue[current].cutPrice(percent);
80              current = (current + 1) % queue.length;
81          }
82      }
83  }
84
```

3.6 Клас WarehouseManager.java

```java
/*
 * Copyright (c) 2026. Anastasiia Gorbunova
 * National University "Odesa Polytechnic"
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://apache.org
 */
package ad251.gorbunova;

/**
 * Клас для виконання індивідуального завдання №7 "Облік товарів на складі".
 * Оперує структурами масивів для аналізу залишків та пошуку дефіцитних позицій.
 *
 * @author Анастасія Горбунова
 * @version 2.0 09 Oct 2026
 */
public class WarehouseManager {

    /**
     * Запускає повний аналіз складських залишків.
     * Визначає загальну кількість, лідера за обсягом та дефіцитні товари.
     *
     * @param minLimit мінімально допустима кількість одиниць товару на складі
     */
    public static void analyzeWarehouse(int minLimit) {
        String[] itemNames = {"Процесор", "Материнська плата", "ОЗП", "SSD Диск", "Блок живлення"};
        int[] itemQuantities = {15, 8, 42, 24, 5}; /* Паралельний масив кількості одиниць */

        int maxIndex = 0;            /* Індекс товару з максимальною кількістю */
        int totalItems = 0;            /* Загальна сума всіх одиниць на складі */

        System.out.print("Всі товари на складі:\n");
        for (int i = 0; i < itemNames.length; i++) {
            System.out.printf("  - %-20s: %d од.\n", itemNames[i], itemQuantities[i]);
            totalItems += itemQuantities[i];

            // Пошук індексу максимального елемента масиву
            if (itemQuantities[i] > itemQuantities[maxIndex]) {
                maxIndex = i;
            }
        }

        System.out.print("\nРезультати аналізу складу за Варіантом №7:\n");
        System.out.printf("  1) Загальна кількість товарів на складі: %d од.\n", totalItems);
        System.out.printf("  2) Товар із найбільшою кількістю: %s (%d од.)\n", itemNames[maxIndex],
        ↪  itemQuantities[maxIndex]);

        /* Перевірка умовних критеріїв для пошуку дефіцитних позицій */
        System.out.printf("  3) Товари, кількості яких менше за ліміт (%d од.):\n", minLimit);
        for (int i = 0; i < itemNames.length; i++) {
            if (itemQuantities[i] < minLimit) {
                System.out.printf("     * %s (залишилось лише %d од.)\n", itemNames[i],
                ↪  itemQuantities[i]);
            }
        }
    }
}
```

4 РЕЗУЛЬТАТИ РОБОТИ ПРОГРАМИ

Для верифікації правильності роботи розроблених алгоритмів, контролю стану об’єктів та перевірки механізмів інкапсуляції було здійснено запуск програми в єдиному циклі збирання середовища VS Code. На рисунку 4.1 представлено фінальний вивід результатів у термінал Windows PowerShell.

```
--- РЕЗУЛЬТАТ ВВИКОНАННЯ ---
=== Лабораторна робота №2 ===

Тема: Основи роботи з класами та об'єктами.
Інкапсуляція.
Модифікатори доступу.

--- Тестування Кошика на базі СТЕКА (LIFO) ---
Початкова сума товарів у стеку: 36300,00 грн
Сума після підвищення цін на 15%: 41745,00 грн
Сума після зниження цін на 30%: 29221,50 грн

--- Тестування Кошика на базі ЧЕРГИ (FIFO) ---
Початкова сума у черзі: 27100,00 грн
Сума у черзі після підвищення на 15%: 31165,00 грн

--- Індивідуальне завдання (Варіант №7) ---
Всі товари на складі:
  - Процесор            : 15 од.
  - Материнська плата   : 8 од.
  - ОЗП                 : 42 од.
  - SSD Диск            : 24 од.
  - Блок живлення       : 5 од.

Результати аналізу складу за Варіантом №7:
  1) Загальна кількість товарів на складі: 94 од.
  2) Товар із найбільшою кількістю: ОЗП (42 од.)
  3) Товари, кількості яких менше за ліміт (10 од.):
     * Материнська плата (залишилось лише 8 од.)
     * Блок живлення (залишилось лише 5 од.)
```

Рисунок 4.1 – Вигляд терміналу після успішного виконання та тестування програми.

На основі отриманих даних терміналу (рис. 4.1) проведено покроковий аналіз виконання технічного завдання:

1) **Тестування Кошика на базі Стека (`StackCart`):** При ініціалізації п’ятьма базовими товарами загальна сума склала `36300.00 грн`. Після надсилання об’єктам

повідомлення `raiseAllPrices(15.0f)` сума успішно збільшилась на 15% і склала `41745.00 грн`. Подальше зниження цін усіх товарів у стеку на 30% через метод `cutAllPrices(30.0f)` продемонструвало коректну зміну стану об'єктів — фінальна сума зафіксована на рівні `29221.50 грн`.

2) **Тестування сокриття реалізації (`QueueCart`):** Зміна внутрішньої структури даних на кільцеву чергу (`FIFO`) при повному збереженні зовнішнього інтерфейсу методів підтвердила успішність застосування інкапсуляції. Об'єкти додавалися та розраховувалися безпомилково, видаючи аналогічні математичні результати для початкових тестів (`27100.00 грн` та `31165.00 грн` після підвищення цін).

3) **Аналіз складського обліку (Індивідуальний Варіант №7):** Компонент `WarehouseManager` провів лінійний аналіз паралельних масивів даних. Програма безпомилково розрахувала загальну кількість одиниць техніки на складі (`94 од.`), визначила лідера за обсягом залишків (товар «ОЗП» — `42 од.`) та здійснила фільтрацію дефіцитних позицій за заданим лімітом у `10 од.`, вивівши у консоль товари «Материнська плата» (`8 од.`) та «Блок живлення» (`5 од.`).

5 ВІДПОВІДІ НА КОНТРОЛЬНІ ПИТАННЯ

У цьому розділі наведено теоретичні відповіді на контрольні питання з метою підтвердження засвоєння фундаментальних концепцій об’єктно-орієнтованого програмування.

1. Що таке програма з погляду об’єктно-орієнтованої парадигми?

З погляду об’єктно-орієнтованої парадигми (ООП), програма є **сукупністю взаємодіючих об’єктів**, кожен з яких є екземпляром певного класу. Замість послідовного виконання команд (як у процедурному підході), виконання програми базується на надсиланні повідомлень між об’єктами та зміні їхніх внутрішніх станів.

2. Що таке об’єкт?

Об’єкт — це конкретний будівельний блок об’єктно-орієнтованої програми, який є **реальним екземпляром абстрактного класу** та поєднує в собі фіксовані дані (стан) і методи для їхньої обробки (поведінку).

3. З яких частин складається об’єкт?

Кожен об’єкт в ООП складається з трьох обов’язкових частин:

– **Ім’я (ідентифікатор):** унікальна назва або посилання на об’єкт у пам’яті програми (наприклад, посилання `stackCart`).

– **Стан (дані/атрибути):** поточні значення змінних екземпляра, які характеризують властивості об’єкта в даний момент часу.

– **Поведінка (методи):** набір операцій, які об’єкт може виконувати або які можна виконати над ним.

4. Що таке клас?

Клас — це **абстрактний шаблон або креслення**, на основі якого створюються реальні об’єкти (екземпляри). Клас визначає новий тип даних, описую-

чи структуру полів та алгоритми методів, що будуть спільними для всіх його об'єктів.

5. З яких частин складається клас?

Тіло класу складається з двох основних частин, які узагальнено називаються **членами класу**:

– **Поля класу (змінні екземпляра):** визначають структуру даних об'єкта.

– **Методи класу (функції):** містять програмний код для обробки полів та визначають зовнішній інтерфейс взаємодії.

6. Як створити об'єкт якогось класу?

У мові Java об'єкт створюється динамічно у кучі (*Heap*) за допомогою **оператора `new`**, після якого викликається конструктор відповідного класу (наприклад, `Cart cart = new StackCart(8);`). Оператор `new` виділяє пам'ять під об'єкт і повертає посилання на нього.

7. Що таке конструктор? Що таке конструктор за замовчуванням?

Конструктор — це спеціальний метод класу, який автоматично викликається під час створення об'єкта для ініціалізації його початкового стану. Його ім'я збігається з ім'ям класу, і він не має типу повертаємого значення [1].

Конструктор за замовчуванням — це конструктор без параметрів, який автоматично створюється компілятором Java, якщо в класі не оголошено жодного кастомного конструктора. Він ініціалізує числові поля нулями, логічні — значенням `false`, а посилання — значенням `null`.

8. Що таке інкапсуляція? Навіщо вона потрібна? Як реалізовано інкапсуляцію в м

Інкапсуляція — це принцип ООП, що полягає в об'єднанні даних та методів їхньої обробки всередині об'єкта, а також у **прихованні внутрішньої реалізації** від прямого зовнішнього втручання [1].

Навіщо потрібна: вона захищає дані від некоректної модифікації, ізолює кон-

тракт взаємодії від внутрішнього устрою, що дозволяє змінювати код всередині класу (наприклад, замінити стек чергою) без збоїв у решті програми [1].

Реалізація в Java: забезпечується за допомогою **модифікаторів доступу** (`privat` `public`, `protected`). Поля класу оголошуються як `private`, а доступ до них та валідація контролюються через відкриті (`public`) методи — геттери, сеттери та кастомні методи бізнес-логіки.

6 ВИСНОВКИ:

В ході виконання лабораторної роботи було успішно досягнуто поставлену мету:

На основі виконаних досліджень та розробленого програмного комплексу сформульовано такі науково-практичні висновки:

1) **Ознайомились з основами написання класів:** Спроектовано та розподілено по окремих файлах базові класи сутностей (`Item`), класи логічних структур (`StackCart`, `QueueCart`), сервісні компоненти (`WarehouseManager`) та головний оркестратор (`Main`) згідно з вимогами *Oracle Code Conventions*.

2) **Освоїти інкапсуляцію класів:** Всі внутрішні змінні станів (поля `price`, `name`, масиви `stack` та `queue`) повністю ізольовані від зовнішнього втручання. Доступ до них та валідація даних (наприклад, примусове скидання негативної ціни в 0) реалізовано виключно через публічний інтерфейс методів та геттерів/сеттерів.

3) **Розібралися із використанням конструктора:** Для всіх класів розроблено кастомні конструктори з параметрами (ініціалізація об’єктів товарів та виділення пам’яті під масиви заданої ємності). Для розв’язання конфліктів імен між вхідними аргументами та полями класу у кучі (*Heap*) успішно застосовано ключове слово `this`.

7 ІНФОРМАЦІЙНІ РЕСУРСИ ТА ПОСИЛАННЯ НА ВИХІДНИЙ КОД

Згідно з вимогами до здачі лабораторної роботи, повний проект вихідного коду розробленого додатку розміщено у віддаленому репозиторії системи контролю версій.

– **Посилання на Git-репозиторій проєкту:**
`https://github.com/nasuehfw/oop-lab2.git`

– **Електронний каталог спільного доступу:**
https://drive.google.com/drive/folders/1mIulRQSfoHAyQ2GnqdQbwGz2dFX-5VGK?usp=drive_link